

Distributed Frames with a Fast Remote Node: Collection Becomes the Wall, Batching Removes It

Matias Saavedra

Wednesday 15th July, 2026

Binary: branch feat/frame-distribution, code state c5bd3ed (= the report-26 code, ffe61aa+8c21bf4 lineage, docs-only commits atop it) on *both* nodes. Build portability for the new node: commit c8ac920 (Makefile toolchain autodetection — no source change). Collection batching: commit 1b5da5f (scripts/dist_frames.sh, scripts/dist_dynamic.sh — orchestration only, binary untouched). Benchmarks and data: commit f310656. Raw data: experiments/25-frame-distribution/ (results_bench_yeco.csv, results_dynamic_yeco.csv, results_dynamic_yeco_scp.csv).

Resumen

Reports 25–26 measured frame distribution on a pair whose remote node is $4.56\times$ slower than the laptop. This report reruns both protocols on the mirrored pair: **yeco** (Core Ultra 7 265K, 20 threads, RTX 4090, reached through an SSH jump host) is $4.1\times$ faster than the laptop (solo 5.15 ± 0.10 vs. 21.02 ± 1.94 s). The role reversal reproduces every report-25 conclusion from the other side — equal shares strand the slow node, block orientation is a crude 70/30 weighting, the weighted ideal is $1.24\times$ over the fast node alone — and then exposes something the LAN pair never could: **with weights calibrated (1:4), both nodes compute at the 4.14 s harmonic ideal, yet the end-to-end wall was 13.11 ± 1.81 s — worse than equal shares.** The missing ~ 8 s is *collection*: per-file scp round trips (~ 68 ms/file at ~ 25 ms RTT) that equal-share configs hide behind the stranded node’s compute and balanced configs expose. A controlled transfer A/B on the identical 79-file/36 MB share measures scp at 6.60 s vs. a single tar stream at 0.96 s ($6.9\times$, byte-identical). Batching collection into one tar stream per rank drops weighted static to **6.45 ± 0.26 s** (-51% , $\sigma \times 7$ tighter) — parity with yeco-solo-plus-fetch, the capacity-not-latency ceiling realized for the third time. Dynamic stealing self-schedules on *lane time* (compute + shipping): under scp the *slow* node steals 22 frames from the fast one and lands at the lane-equalizing optimum; after batching, right weights make stealing a $+21\%$ chunking tax, and wrong weights reveal the $KCAP \cdot w_r$ chunk cap throttling the fast node — the mirror image of the in-flight defect report 26 fixed.

1. Motivation: the mirrored pair

The laptop+ivy campaign left two loose ends. First, its conclusions were all derived on a pair where the *remote* node is the slow one — ivy at 0.855 s/frame vs. the laptop hybrid at 0.188 — so “equal shares strand the wall on the slow node” had only been observed with the slow node conveniently at the far end of the SSH link. Second, both nodes sat on the same LAN, which kept orchestration overhead at a flat 3.3–4.2 s and made the transport invisible in every analysis. A new node inverts both conditions at

once: **yeco** renders the production spec $4.1\times$ faster than the laptop *and* sits behind a WAN jump host. This report brings the node up (a Makefile portability fix, **S2**), reruns the report-25 equal-share sweep and the report-26 weighted/dynamic A/B on the new pair, diagnoses the transport bottleneck the reruns expose, fixes it (one tar stream per rank, commit **1b5da5f**), and re-measures.

2. Setup

Nodes.

Node A is the usual laptop (i7-9750H, 6C/12T + GTX 1660 Ti Max-Q, Arch, g++-15, CUDA 13.2, on AC for every timed run; args **12 1 0.1 32768 1 1**). Node B is **yeco**: Intel Core Ultra 7 265K (20 threads), RTX 4090 (sm_89, 24 GB), Ubuntu 24.04, CUDA 13.3 at **/usr/local/cuda**, g++ 13.3; a desktop, so no battery gate applies (args **20 1 0.1 32768 1 1**). Both nodes run the same code state (**c5bd3ed**); each builds natively with plain **make** after commit **c8ac920**, which replaced the Makefile’s hard-coded Arch toolchain (g++-15, **/opt/cuda**) with overridable autodetection: **CUDAINST** **!=** first existing of **/opt/cuda**, **/usr/local/cuda**; **HOSTCC** **!=** **g++-15** if present else **g++**; and **nvcc** invoked as **\$(CUDAINST)/bin/nvcc** — never **PATH**’s, because Ubuntu ships a stale apt CUDA 12.0 in **/usr/bin** whose **nvcc** rejects gcc 13.

Network.

yeco is reachable only through a ProxyJump (**laptop** $\rightarrow$ **dichato.dcc.uchile.cl** $\rightarrow$ **yeco**), configured as an **ssh yeco** alias with connection multiplexing (**ControlMaster/ControlPersist**), so no run pays per-connection handshakes. Measured link properties (warm master): command round trip ~ 25 ms; single-file bulk transfer $\sim 30\text{--}37$ MB/s (20 MB in 0.66 s). Both numbers matter in **S5.3**.

Workload and protocol.

Production spec (100 frames, 1920×1080 , the $32\times$ cost-trend zoom), **diffT**=0.1, **quiet**=1, **save**=1, 3 reps per config, same-day in-batch baselines — identical to reports 25/26. **wall_e2e_s** times the whole orchestration (ship spec + ranks + collect); each node’s own **[total_elapsed_s]** is its compute wall (includes local PNG encode/write, excludes network).

Identity scope.

The benchmarks verify the *union* (100/100 distinct global frames collected per run). Byte-identity hybrid-vs-hybrid across this pair is *not expected and not claimed*: sm_75+nvcc 13.2 vs. sm_89+nvcc 13.3 can differ in last-ulp FP64 rounding, the same mechanism as the report-20 CPU-vs-CUDA differences. Report 25’s byte-identity guarantee remains scoped to executor-pinned CPU runs under the plain **-O2** rule; that check was not re-run on this pair.

3. The scripts

Four scripts produced every number in this report. The two orchestrators are report-25/26 infrastructure, recapped here because the transport fix lands inside them; the two benchmark drivers are new.

3.1. scripts/dist_frames.sh — static distribution

One invocation renders the whole spec across N nodes: a hosts file gives one **host** [**rdir** [**args...**]] line per rank (":" = this machine without SSH), and GNU parallel runs one per-rank job each of which (1) creates **rdir/dist_work/rank r** on its host, (2) ships the spec there, (3) runs the binary with **DIST_NODES/DIST_RANK/DIST_BLOCK** (or **DIST_WEIGHTS**, precedence **DIST_FRAMES** > **DIST_WEIGHTS** > block-cyclic) selecting that rank's frames inside **main.cpp**'s frame loop, and (4) copies the rank's PNGs back to the collection directory. Because output filenames carry the *global* frame index, collection is a plain union — no renumbering, no collisions. Step (4) is where this report's fix lives (§3.4).

3.2. scripts/dist_dynamic.sh — stealing coordinator

The report-26 dynamic mode: the frame list is shuffled with a seeded deterministic shuffle and dealt into per-rank bags proportionally to **WEIGHTS**; one driver per node then dispatches its bag in guided chunks — half the remaining bag, capped at $\text{KCAP} \cdot w_r$ frames (default **KCAP** = 8), min **KMIN** — as repeated binary invocations with **DIST_FRAMES**=**<list>**. A driver whose bag empties *steals* half the richest node's undischarged tail; all bag mutations are serialized under one **flock**, and in-flight chunks are already out of the bags, so every frame renders exactly once. Two properties matter below: each chunk is a fresh process (so a GPU node pays CUDA-context creation per chunk), and each chunk's PNGs are copied back *inside the driver loop, between the chunk and the next take_chunk* — so a node's dispatch cadence includes its shipping.

3.3. bench_2node_yeco.sh and bench_dynamic_yeco.sh

Faithful ports of **bench_2node.sh** / **bench_dynamic.sh** (same configs, same CSV schema, same in-batch baseline discipline) with three deliberate differences. (1) The remote node is **yeco** with args **20 1 0.1 32768 1 1** — a *hybrid* node, unlike **ivy**'s CPU-only build. (2) The AC gate checks only the laptop: **yeco** has no battery to check. (3) **yeco_solo** runs 3 reps instead of **ivy**'s 1 — at ~5 s it is too cheap not to. The dynamic driver uses weights **1:4** (laptop:yeco), the measured throughput ratio ($21.02/5.15 = 4.08$), plus the deliberately wrong **1:1** row — the mirror of report 26's 5:1/1:1 pairing. The full A/B was run twice: once with the original per-file scp collection (**results_dynamic_yeco_scp.csv**, preserved per the **results_dynamic_uncapped.csv** precedent — it motivates the fix) and once after the batching commit (**results_dynamic_yeco.csv**).

3.4. The transport fix (commit 1b5da5f)

Código 1: scripts/dist_frames.sh — rank collection, before and after. The same two-line change lands at the per-chunk copy in **dist_dynamic.sh**.

```
1 # before: one scp invocation -- but the SCP/SFTP protocol still moves
2 # files ONE AT A TIME, paying 2-3 protocol round trips per file
3 scp -q "$host:$wd/${DIST_PREFIX}*.png" "$DIST_OUT/"
4
```

```

5 # after: one tar stream -- files packed back-to-back on a single pipe,
6 # TCP stays full; -m = don't restore mtimes (matches scp's default and
7 # silences "time stamp in the future" from small clock skew)
8 ssh "$host" "cd '$wd' && tar cf - ${DIST_PREFIX}*.png" | tar xmf - -C "
    ↪ $DIST_OUT"

```

No compression flag: PNGs are already deflate-compressed, so `-z` would only burn CPU on the critical path. The local (":") branch keeps its `cp`.

4. Results

Tabla 1: Equal-share static sweep (`results_bench_yeco.csv`), means $\pm \sigma$ over 3 reps, per-file scp collection. “laptop/yeco” are each node’s own compute wall; “exposed overhead” = e2e minus the slower rank’s compute — the part of shipping/orchestration *not* hidden behind a still-computing node. Δ compares against the in-batch laptop baseline; every 2-node config wins against the laptop and loses against yeco solo (5.15 s). “block” sends the deep (expensive) half of the zoom to yeco; “block rev.” sends it to the laptop.

Config	e2e (s)	laptop (s)	yeco (s)	exp. ovh. (s)	Δ
laptop hybrid (baseline)	21.02 ± 1.94	21.02	—	—	—
yeco solo	5.15 ± 0.10	—	5.15	—	−75%
2node cyclic (block=1)	11.24 ± 0.18	10.97	2.71	0.27	−47%
2node block-cyclic(8)	12.23 ± 0.90	11.80	2.58	0.43	−42%
2node block	11.39 ± 2.05	6.64	3.43	4.75	−46%
2node block rev.	14.75 ± 0.34	14.17	1.83	0.59	−30%

Tabla 2: Transfer micro-A/B on the identical payload (yeco’s 79-PNG / 36 MB weighted-static share, warm multiplexed connection; both transfers md5-identical file sets). The link is not the problem — the per-file protocol is.

Method	Time (s)	Effective rate	Per-file cost
scp of the glob (79 files)	6.60	5.5 MB/s	~68 ms
ssh tar single stream	0.96	37 MB/s	~0
single 20 MB file (reference)	0.66	30 MB/s	—
command round trip (ssh yeco true)	0.025	—	—

Tabla 3: Weighted/dynamic A/B under both transports, means $\pm \sigma$ over 3 reps (data: `results_dynamic_yeco_scp.csv` and `results_dynamic_yeco.csv`). “laptop/yeco” are compute walls (static: the rank’s `[total_elapsed_s]`; dynamic: sum over its chunks); “shares” are frames rendered, stolen frames in parentheses (under scp the *laptop* steals; the meaning of that inversion is $\mathcal{S}5.5$). “L+22” means the laptop stole 22 frames. The scp and tar runs are separate same-day batches, each with its own baseline.

Transport	Config	e2e (s)	laptop (s)	yeco (s)	shares L/Y	Δ
per-file scp	baseline	20.75 ± 2.06	20.75	—	100/—	—
	weighted 1:4	13.11 ± 1.81	4.56	4.11	21/79	−37%
	dyn 1:4	12.95 ± 3.45	8.93	3.89	42/58 (L+22)	−38%
	dyn 1:1 (wrong)	15.17 ± 2.01	10.92	4.12	45/55 (Y+5)	−27%
tar stream	baseline	20.07 ± 1.51	20.07	—	100/—	—
	weighted 1:4	6.45 ± 0.26	4.21	4.19	21/79	−68%
	dyn 1:4	7.78 ± 0.81	6.24	4.46	32/68 (L+12)	−61%
	dyn 1:1 (wrong)	10.90 ± 1.58	8.80	4.81	37/63 (Y+13)	−46%

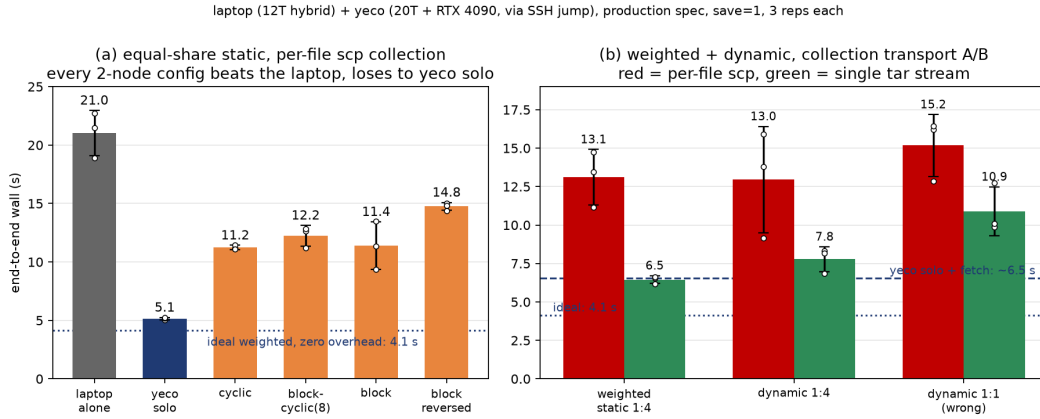


Figure 1: Both campaigns (bars: 3-rep means, whiskers σ , dots reps). Panel (a): every equal-share config (orange) beats the laptop (grey) and loses to yeco solo (blue); “block” gives yeco the deep half of the zoom, “block reversed” gives it to the laptop. Panel (b): the transport A/B — red (per-file scp) vs. green (tar stream) on identical schedules. The green weighted-static bar sits on the dashed yeco-solo-plus-fetch line: parity, the ceiling this pair allows. The dotted line is the zero-overhead harmonic ideal (4.1 s).

Reference arithmetic used throughout the analysis: per-frame cost 0.210 s (laptop) vs. 0.0515 s (yeco solo, save=1), ratio 4.08; harmonic ideal $1/(1/21.02 + 1/5.15) = 4.14$ s = $1.24\times$ over yeco solo — almost the same ceiling as ivy’s $1.22\times$, by coincidence of the similar speed gaps.

5. Analysis

5.1. The roles flip on cue: equal shares strand whichever node is slow

Every equal-share config beats the laptop baseline by 30–47% (Table 1) — and every one loses to yeco solo by $2.2\times$ or more. Both statements are the same mechanism report 25 measured from the other side: under equal shares each node gets ~ 50 frames, the laptop needs ~ 11 s for its half where yeco needs 2.6–3.4 s, so the wall is the laptop’s share and yeco idles $\sim 75\%$ of the run. Block orientation replays its ivy role with the sign flipped: the zoom’s cost trend puts $\sim 70\%$ of the work in the deep half, so “block” (deep half to the *fast* node) cuts the laptop’s compute to 6.64 s — the built-in crude weighting, now pointing the right way — while “block reversed” inflates it to 14.17 s. That a $4\times$ -faster remote node cannot beat its own solo time under *any* equal-share schedule is the report-25 law verbatim: the slow node in the pair — whichever side of the SSH link it sits on — buys capacity, not latency.

5.2. The overhead ledger: better balance exposes the transport

The e2e wall of a 2-node run decomposes as $\max(C_\ell, C_y + S_y)$ plus small fixed costs: the laptop’s collection is a local **cp** ($S_\ell \approx 0$), while yeco’s share must cross the WAN *after* its compute (S_y), serially within its rank job (**S3.1**). The “exposed overhead” column of Table 1 is this structure made visible: cyclic, block-cyclic and reversed block all leave the laptop computing for 11–14 s, far past yeco’s 2–3 s finish — yeco’s shipping completes in that shadow, and the measured exposure is only 0.27–0.59 s. The “block” config is the outlier that gives the game away: the laptop finishes in 6.64 s, the shadow shrinks, and 4.75 s of shipping suddenly appears in the e2e — with σ jumping to 2.05 s, WAN jitter now on the critical path.

The weighted run completes the demonstration. With 1:4 shares both nodes compute at the harmonic ideal — laptop 4.56 s, yeco 4.11 s against the analytic 4.14 — **and the e2e is 13.11 ± 1.81 s, worse than equal-share cyclic’s 11.24**. Perfect balance removed the last of the shadow: yeco’s 79-file shipping (~ 8 s under scp) stands fully exposed. The general statement, and the reason this never surfaced on the LAN pair: *scheduling quality and transport visibility trade off — the closer the schedule gets to the compute ideal, the more of the collection cost lands on the critical path*. On ivy the same structural exposure existed (ivy finished last and shipped last) but cost a flat 3–4 s against 40–60 s walls; here it is ~ 8 s against a 4 s compute ideal — 200% of the prize.

5.3. Diagnosis: latency, not bandwidth — 68 ms per file

Table 2 pins the mechanism with one controlled A/B on the identical payload. The link moves a single 20 MB file at 30 MB/s and the tar stream at 37 MB/s, yet scp needs 6.60 s for 36 MB spread over 79 files — an effective 5.5 MB/s. The arithmetic closes exactly: $6.60 - 1.0$ s of actual bytes = 5.6 s of protocol over 79 files ≈ 68 ms/file ≈ 2 –3 protocol round trips at the measured 25 ms RTT — scp transfers files strictly one at a time, and each file pays its open/write/close exchanges before the next starts. The tar pipe packs the same files back-to-back on one stream: zero per-file

round trips, line rate, byte-identical delivery (md5-verified). In-run the effective per-file cost is a little higher (4.75 s/50 files = 95 ms in the block config’s exposure) — the micro-benchmark’s warm best case plus jitter. Neither weights nor stealing can touch this term; it scales with the fast node’s *share*, which good scheduling *increases*.

5.4. **Batching realizes the ceiling: 6.45 ± 0.26 s, parity with solo-plus-fetch**

The identical weighted-static schedule drops $13.11 \rightarrow 6.45$ s (−51%) with the transport change alone, and σ collapses $1.81 \rightarrow 0.26$ s — a $7\times$ tightening that retroactively identifies the pre-batch variance as per-file WAN jitter, not compute noise. The residual decomposes cleanly: $\max(4.21, 4.19 + 1.0) + \text{spec ship} + \sim 1.2$ s of fixed orchestration (GNU parallel spin-up, per-rank `mkdir/cleanup/ownedFrames` round trips) ≈ 6.4 s — what was measured. Against the alternatives: $3.1\times$ the laptop baseline, and *parity* with fetching a yeco solo run’s output ($5.15 + 1.4 \approx 6.5$ s) — the delivered-to-laptop metric that is this workflow’s honest deliverable. For the third time (ivy static, ivy dynamic, now yeco) the pair lands exactly at its capacity-not-latency ceiling: the slow partner cannot shorten the wall, and with the transport fixed it finally stops lengthening it. The tightest-variance distributed config this project has measured is now also its fastest delivered result.

5.5. **Dynamic stealing schedules lane time — which inverts the steal direction**

Under scp, `dyn_14`’s final split was 42/58 with the *laptop* stealing 22 frames from the bag of a node $4\times$ faster — backwards, by compute logic. It is correct by *lane* logic: each chunk’s shipping sits between that chunk and the driver’s next `take_chunk` (`S3`), so a node’s effective rate is compute + shipping, and the empty-bag steal rule equalizes exactly that. Solve $\min \max(0.210 f_\ell, (0.0515 + 0.095) f_y)$ with $f_\ell + f_y = 100$ at the in-run 95 ms/frame shipping cost and the optimum is 41/59 with an 8.6 s wall: the coordinator found 42/58 unprompted, and its best rep hit 9.15 s. The scheduler was optimizing the true cost structure while the static weights optimized the wrong one — but the “right” answer under expensive shipping is to *refuse* the fast node’s capacity, which is treating the symptom.

After batching, the lane costs converge toward pure compute and the steal shrinks on cue (32/68, 12 stolen); the residue is yeco’s remaining per-chunk overhead — a fresh CUDA context plus a dispatch round trip and tar session per chunk, ~ 0.7 s $\times$ 4 chunks on a 3.5 s compute lane. That is also why `dyn_14` now costs +21% over weighted static (7.78 vs. 6.45 s): with calibrated weights stealing never needs to fire, so its price is pure chunking tax. On ivy the same insurance was free (+0–1 s) because ivy’s chunks were CPU-only processes on a LAN, and 47 s of compute dwarfed the dispatch cost. Insurance pricing scales with per-chunk overhead over total compute — cheap for slow CPU nodes, expensive for fast GPU nodes.

5.6. **Wrong weights: the chunk cap now throttles the fast node**

Report 26’s defect was a slow node holding an unstealable 25-frame chunk in flight; the $\text{KCAP} \cdot w_r$ cap fixed it by bounding every node’s in-flight exposure at the same

wall-time budget — *assuming the weights approximate the true rates*. The 1:1 row breaks that assumption in the opposite direction and the cap changes sides: yeco’s cap collapses to $8 \cdot 1 = 8$ frames, forcing **9 dispatches to render 63 frames** it could compute in 3.5 s — each dispatch paying the CUDA context and round trips — while the laptop grinds its own 8-frame chunks at ~ 2 s each. Stealing does improve with batching (5 $\rightarrow$ 13 frames taken, e2e 15.17 $\rightarrow$ 10.90 s), but the recovery is nothing like ivy’s 72%: the fast node reaches its empty-bag steal trigger late (its cadence is overhead-throttled), and in-flight chunks plus the victim’s own steady drain leave only a thin tail to take. The structural statement: $\text{KCAP} \cdot w_r$ bounds exposure in wall time only when $w_r \propto$ true rate — miscalibrated weights corrupt the cap along with the shares. The fix is to stop trusting the operator’s number twice: estimate each node’s rate from its first measured chunk and re-derive caps (and steal sizes) from that.

6. Recommendations

1. **Keep collection batched (done, commit 1b5da5f)**. One tar stream per rank: $6.9\times$ on the transfer itself, -51% on the weighted e2e wall, $\sigma \times 7$ tighter. It also helps the LAN pair (fewer round trips), just less. Trade-off: none measured — delivery is byte-identical and the `-m` flag reproduces scp’s mtime behaviour.
2. **On WAN pairs, use weighted static with measured weights; treat dynamic stealing as paid insurance**. Weighted static 1:4 delivers 6.45 ± 0.26 s; stealing costs $+21\%$ when weights are right (chunking tax on a fast GPU node). Buy it when weights are uncertain; skip it when they are measured.
3. **Weights belong to the (machine, executor-mix, transport) triple**. Report 26 showed executor-mix dependence (5:1 $\rightarrow$ 2.4:1 when CPU-pinned); this report adds transport dependence — under scp the lane-optimal split was 41/59, under tar it is $\sim 24/76$. Recalibrate after any transport change.
4. **Rate-adaptive chunk caps (follow-up)**. Derive each node’s cap and steal size from its first chunk’s measured rate instead of the operator’s weights ($\text{KCAP} \cdot w_r$ is only as good as w_r ; S5.6 shows it throttling a $4\times$ -fast node to 8-frame chunks). Expected effect: the `dyn_11` row approaches the `dyn_14` one without operator input.
5. **If sub-6.5 s delivered matters, the next ~ 1.2 s is fixed orchestration** (parallel spin-up, per-rank `mkdir`/cleanup round trips, spec ship) — amortizable by keeping rank workdirs warm between runs. Below that sits the 4.14 s harmonic ideal, and nothing on this pair goes lower.

7. Conclusion

Rerunning reports 25–26 with the speed roles reversed confirmed the distribution law from the other side — equal shares strand whichever node is slow, and no schedule beats the fast node’s solo throughput — and then surfaced the term the LAN pair had kept invisible: per-file collection latency, ~ 68 ms/file over an SSH jump, ~ 8 s on a 79-file share. It is a cost that grows exactly when scheduling improves, which

is why the compute-ideal weighted split measured *worse* end-to-end than naive equal shares until the transport was fixed. One tar stream per rank removed it: weighted static 1:4 went $13.11 \pm 1.81 \rightarrow 6.45 \pm 0.26$ s, parity with fetching the fast node’s solo output — the pair’s analytic ceiling, realized with the tightest variance of any distributed config measured in this project. The dynamic coordinator, unmodified, priced the same phenomenon from a different angle: it equalizes lane time (compute + shipping), stealing *into* the slow node under scp and shrinking that steal after batching — and with calibrated weights its chunking overhead now makes it strictly insurance, +21% on a fast GPU node where ivy’s CPU chunks had been free. Wrong weights still cost real money because the weight-proportional chunk cap inherits their error; rate-adaptive caps are the identified follow-up. The homogeneous interleave study (report 13’s earmark) remains open — both measured pairs are too asymmetric for interleave effects to be visible.